Continuously Operating Reference Station and High-Precision Satellite Positioning and Navigation Service System

(CORS) User Manual

October 1, 2022

English edition · revised October 2026

Version record

Version	Date	Revision	Editor
V1.0.0	2022/10/01	Manual created	Su Jinglan
V1.0.1	2023/08/05	Revision	Su Jinglan
V1.0.2	2026/10/15	English edition: added Chapter 3 — project structure, code framework and algorithm flows	Su Jinglan

Contents

1 System introduction

1.1 System functions

1.2 System composition

1.2.1 Modules included

1.2.2 Module descriptions

2 Operating instructions

2.1 Configuration files

1. System configuration

2. Reference station input configuration

3. Virtual reference station configuration

4. Baseline RTK solving configuration

5. User configuration

6. Reference station information configuration

2.2 Running the system

2.3 Virtual console

1. Starting the system

2. Adding a reference station

3. Deleting a reference station

4. Loading the system configuration

5. Displaying ephemeris information

6. Displaying satellite observation information

7. Displaying satellite status information

8. Displaying reference station status information

9. Adding a virtual reference station

10. Deleting a virtual reference station

11. Displaying the reference station triangular network

12. Displaying baseline RTK solution information

13. Displaying subnetworks

14. Virtual reference station list

15. Adding a user

16. Deleting a user

17. User list
18. Displaying monitoring users
19. NTRIPAGENT access user list
20. Drawing the Delaunay reference station network
21. Drawing the position relationship between virtual and reference stations
22. Stopping the system
23. Runtime views

3 Project structure, code framework and algorithm flow

3.1 Project structure

3.2 Code framework

3.2.1 Module layering

3.2.2 Threading and concurrency model

3.2.3 Core data structures

3.3 Key algorithm flows

3.3.1 End-to-end data flow

3.3.2 Incremental Delaunay networking

3.3.3 Baseline RTK solving

3.3.4 VRS observation generation

3.4 Build system and third-party components

1 System introduction

1.1 System functions

The continuously operating reference station and high-precision satellite positioning and navigation service system (CORS) is a large-scale integrated positioning and navigation service network built on modern GNSS technology, computer network technology, networked real-time positioning service technology and modern mobile communication technology. The defining feature of CORS is that it brings the concept of networking into geodesy and surveying applications — it not only transforms the surveying and mapping industry, but also introduces a new service model for position and time information in modern society. The system meets a wide range of societal needs in modern information-based management: urban planning, land surveying and mapping, cadastral management, urban and rural construction, environmental monitoring, disaster prevention and mitigation, vessel and vehicle navigation, traffic monitoring, and more. Its main functions are:

- Multi-frequency, multi-constellation network RTK processing: baseline solving, ambiguity closure checks, and more
- Virtual reference station (VRS) technology
- Time synchronization of reference station observation data
- Support for connecting large-scale reference station networks
- Delaunay-triangulation-based reference station networking
- Reference station atmospheric delay error computation
- Integrity monitoring of reference and virtual stations
- Virtual reference station data broadcasting and network RTK differential positioning service

1.2 System composition

1.2.1 Modules included

The system comprises the following modules:

- Reference station data acquisition and decoding module;
- Multi-constellation satellite orbit and clock error computation module;
- Multi-system reference time system conversion module;
- Reference station networking module;
- Reference station baseline solving module;
- Network RTK solving module;
- Virtual reference station pseudorange and carrier phase observation generation module;
- Virtual reference station pseudorange and carrier phase observation broadcasting module;
- Virtual console module.

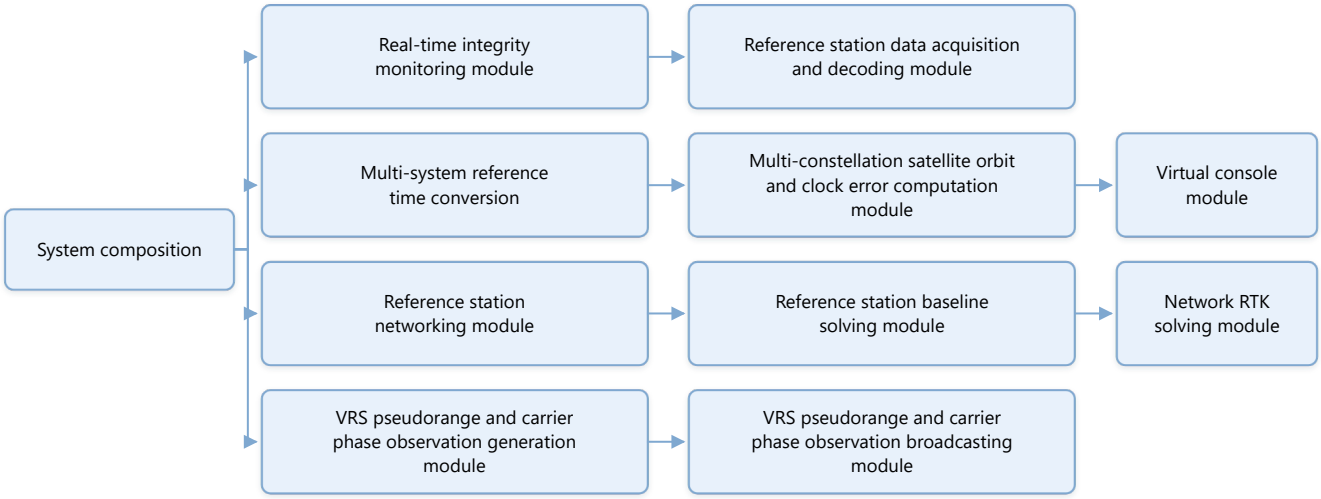


Figure 1.1 System composition

1.2.2 Module descriptions

1. Reference station data acquisition and decoding module

Based on an event-driven asynchronous I/O model, the module connects to physical or virtual stations with higher data processing efficiency and stability, and supports data connection and processing for large-scale physical or virtual stations. It uses the NTRIP and RTCM protocols to connect to continuously operating satellite reference stations and to acquire raw observation data of the BeiDou, GPS, GLONASS, Galileo and QZSS satellite systems, as well as precise coordinates and other information of the reference stations.

The module supports the NTRIP and RTCM protocols. RTCM data is decoded through a queue, so as to obtain the satellite pseudorange and carrier phase observations, Doppler observations, carrier-to-noise ratios and ephemeris data broadcast by the reference stations, as shown in Figure 1.2.

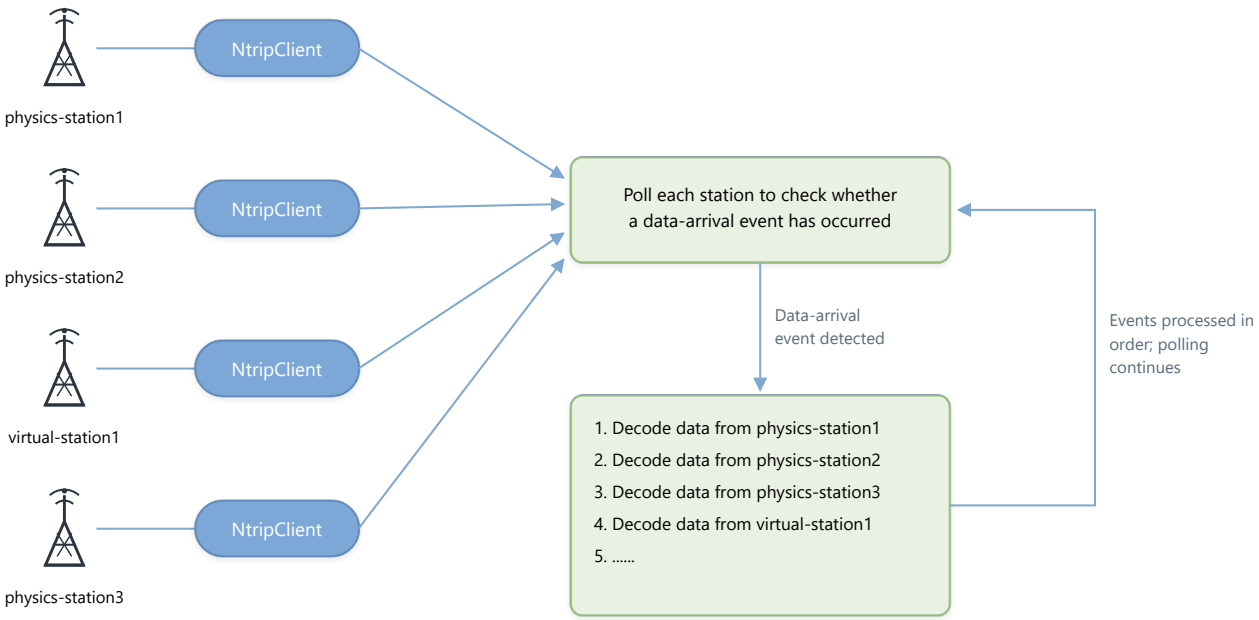


Figure 1.2 Reference station data acquisition and decoding module

2. Multi-constellation satellite orbit and clock error computation module

Using the ephemeris data of each reference station, the system updates the satellite ephemeris database. It computes the orbits and clock errors of BeiDou, GPS, Galileo and GLONASS satellites from the real-time broadcast navigation ephemeris, and detects and excludes anomalous satellites based on the RAIM integrity monitoring algorithm, as shown in Figure 1.4.

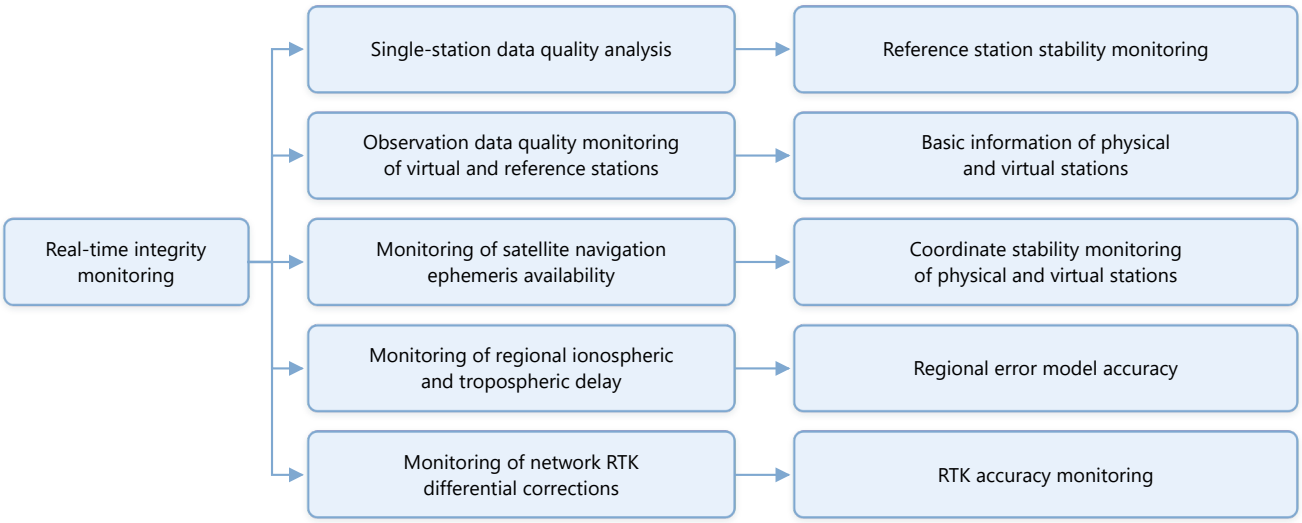


Figure 1.3 Composition of the real-time integrity monitoring module

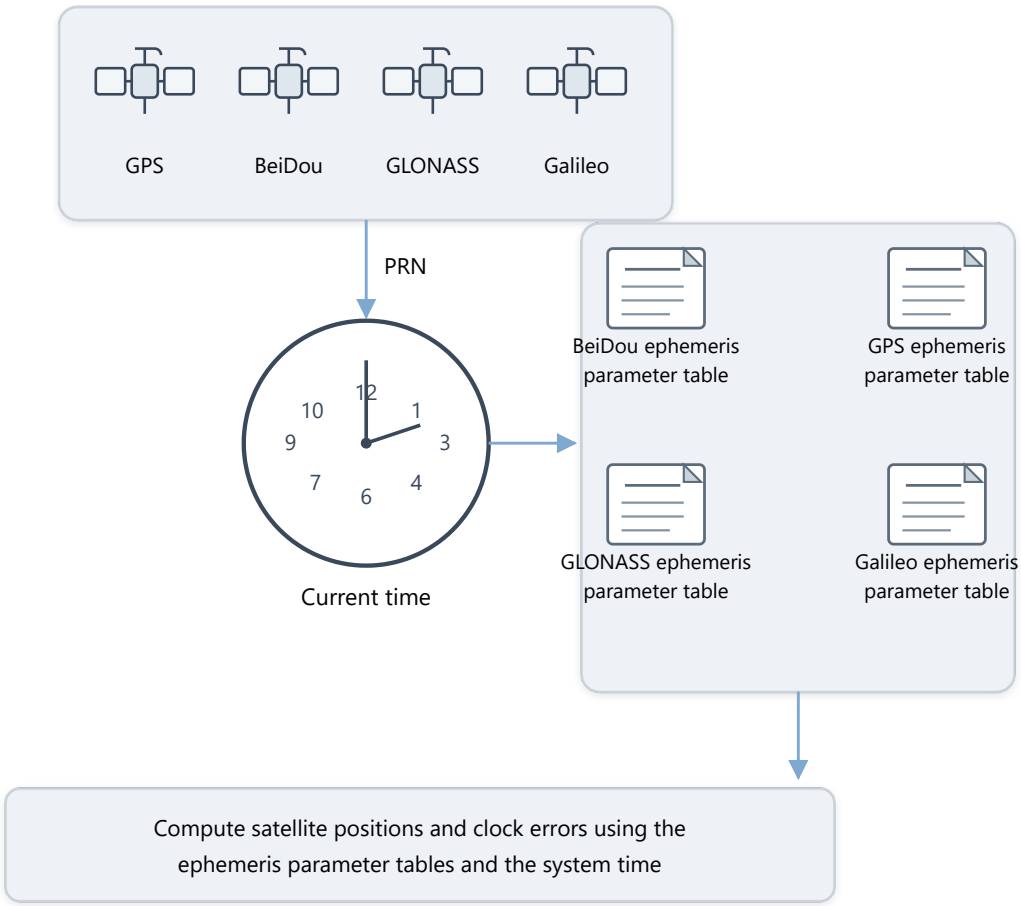


Figure 1.4 Multi-constellation satellite orbit and clock error computation module

3. Multi-system reference time system conversion module

Multi-system reference time system conversion covers the conversion between the BeiDou, GPS, Galileo and GLONASS reference time systems.

4. Reference station networking module

The reference station triangular network is built based on Delaunay triangulation, as shown in Figure 1.5.

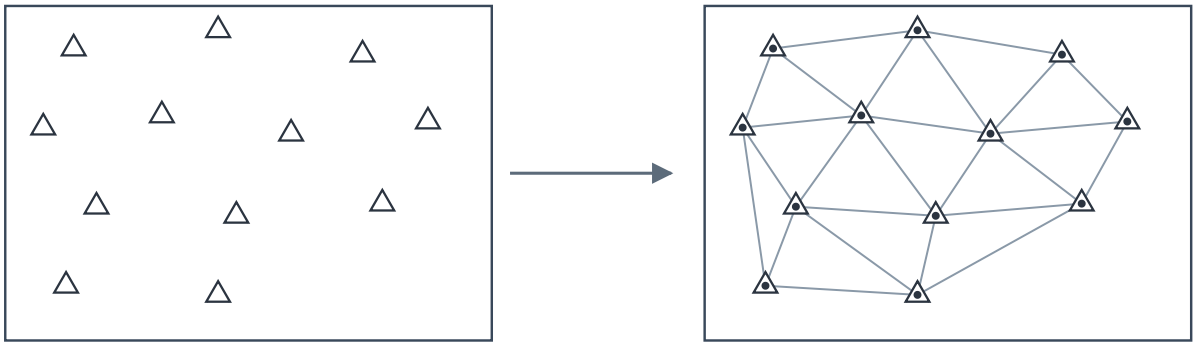


Figure 1.5 Construction of the reference station triangular network

5. Reference station baseline solving module

The module performs RTK solving with the satellite pseudorange and carrier phase observations of the reference stations (supporting BeiDou, GPS, Galileo, GLONASS, QZSS and more), and obtains the inter-station ambiguities and errors (ionospheric, tropospheric and other unmodelled residual errors).

6. Network RTK solving module

A master reference station is selected, and subnetworks are constructed over the reference station triangular network for network RTK solving, as shown in Figure 1.6.

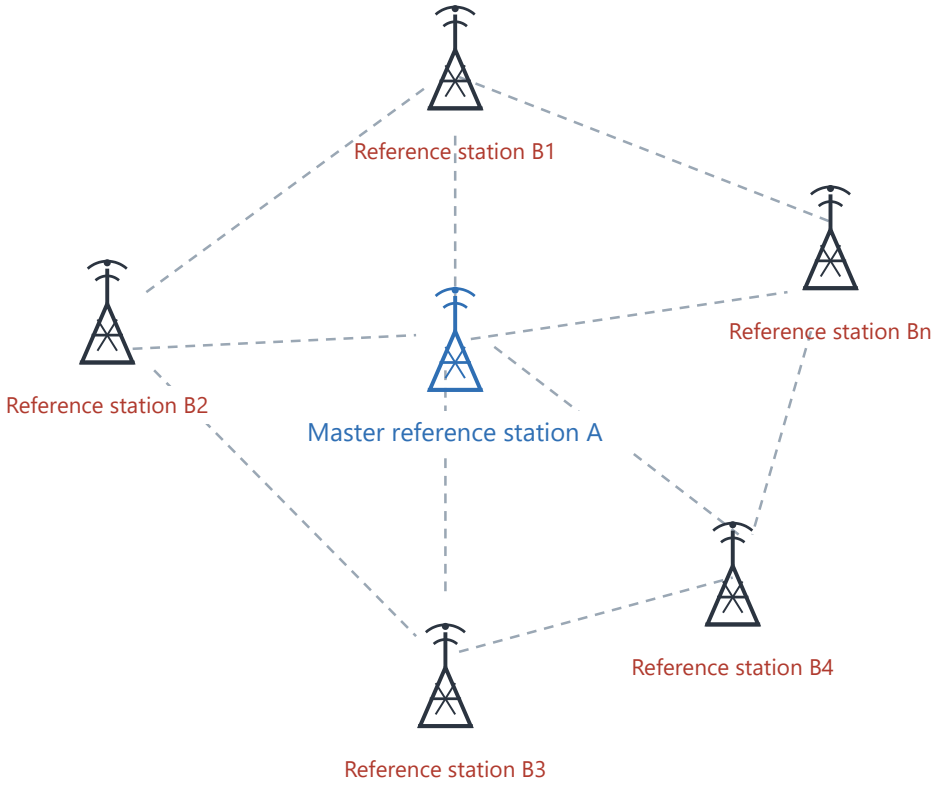


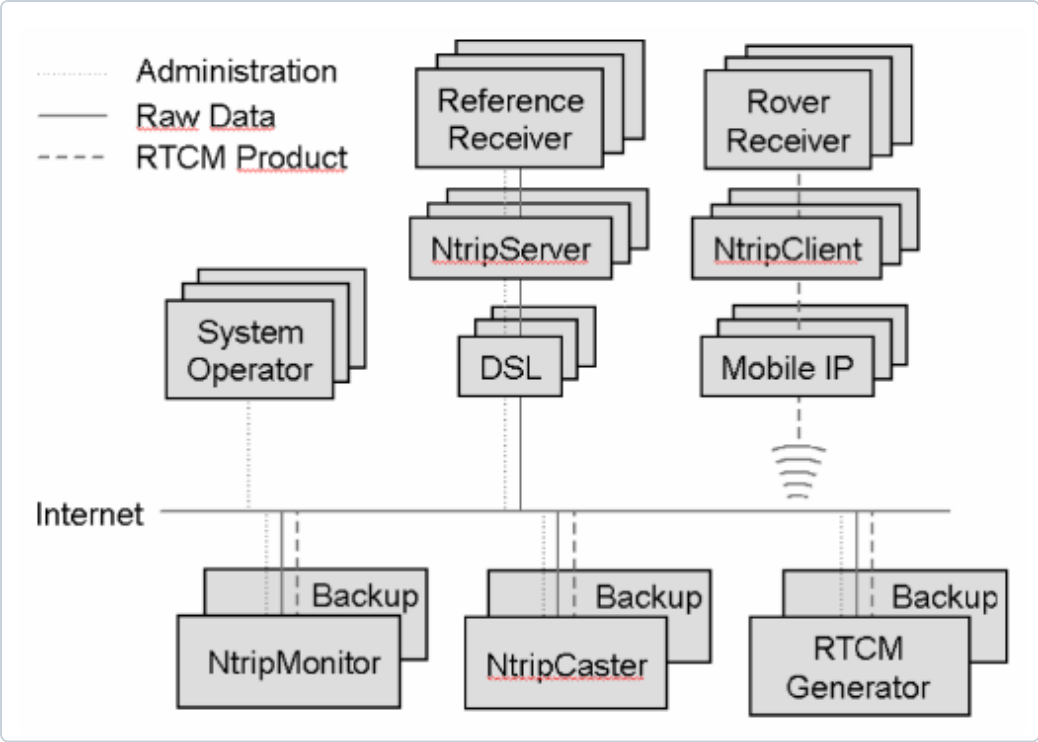
Figure 1.6 Subnetwork construction

7. Virtual reference station pseudorange and carrier phase observation generation module

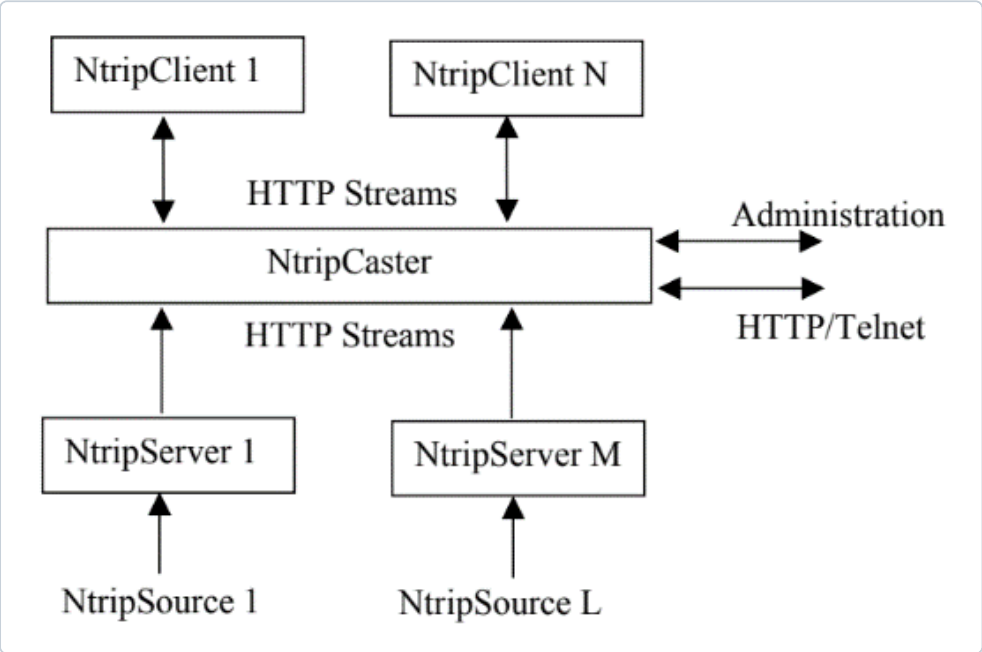
Pseudorange and carrier phase observations are generated according to the position of the virtual reference station.

8. Virtual reference station pseudorange and carrier phase observation broadcasting module

Terminal users connect to the continuously operating reference station and satellite positioning and navigation system through the NTRIP protocol. After successful authentication, the system broadcasts the virtual reference station pseudorange and carrier phase observations to the users via the NTRIP and RTCM protocols, as shown in Figure 1.7.



(a) NTRIP system architecture



(b) NTRIP data flow

Figure 1.7 NTRIP protocol

9. Virtual console module

The module connects to the continuously operating reference station and satellite positioning and navigation system in remote or local mode; commands can be issued to the running system, and the current operating status of the system can be queried through the virtual console, as shown in Figure 1.8.

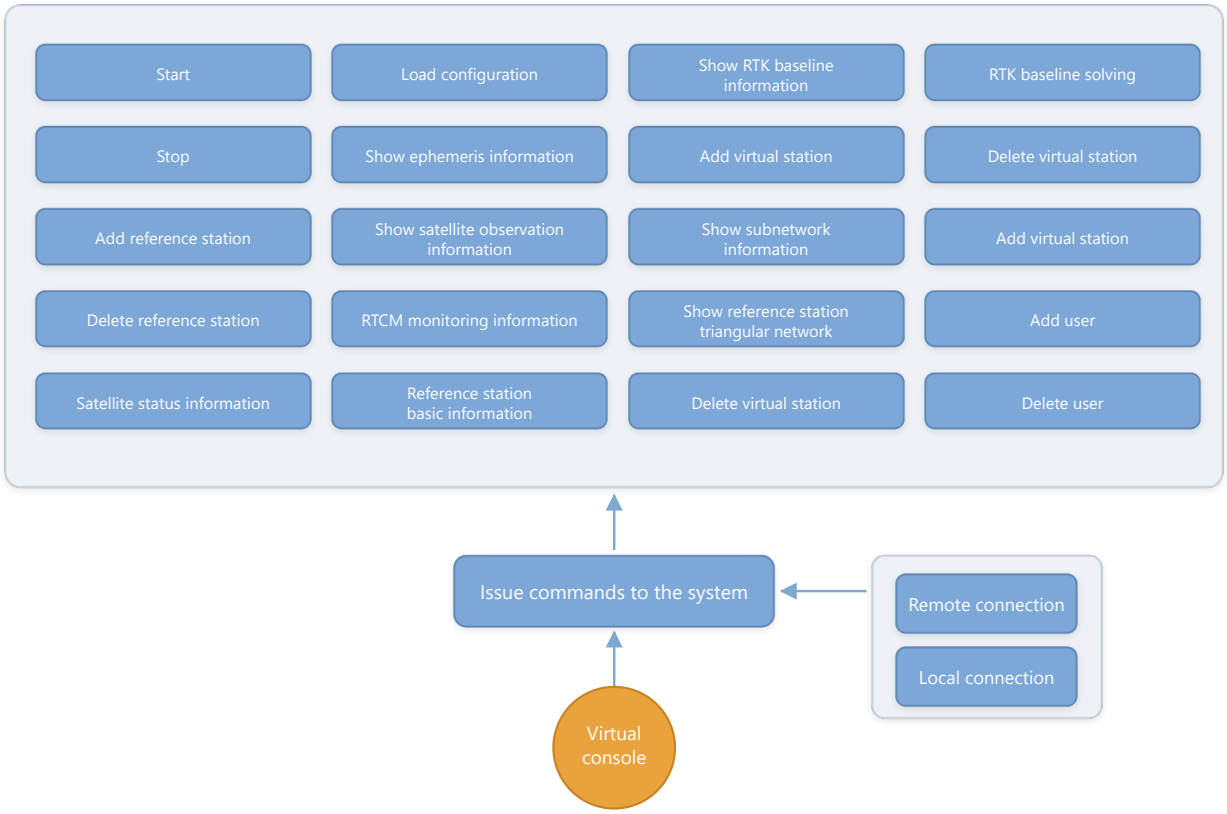


Figure 1.8 Virtual console module

2 Operating instructions

2.1 Configuration files

Before using the continuously operating reference station and high-precision satellite positioning and navigation service system, the configuration files must be prepared.

1. System configuration

Parameter	Description
ntrip-sources-file	Path of the reference station input configuration file
trace-file	Path of the system log file
baselines-file	Path of the baseline list configuration file
base-stations-info-file	Path of the reference station information configuration file
virtual-stations-file	Path of the virtual reference station configuration file
rtk-conf-file	Path of the baseline RTK solving configuration file
agent-user-file	Path of the user configuration file
monitor-port	Monitoring port of the real-time integrity monitoring system

2. Reference station input configuration

Format: station name, IP, port, user name, password, mount point, reference station latitude, reference station longitude, reference station height#

Example:

```
A005,127.0.0.1,8002,test ,pwd ,A005,39.67042106,115.92179510,73.42850#
A006,127.0.0.1,8002,test ,pwd ,A006,39.68042106,115.92379510,73.43850#
A007,127.0.0.1,8002,test ,pwd ,A007,39.64842106,115.92879510,73.45850#
A008,127.0.0.1,8002,test ,pwd ,A008,39.54042106,115.12079510,73.46850#
```

3. Virtual reference station configuration

Format: virtual reference station name, ECEF-X, ECEF-Y, ECEF-Z#

Example:

```
VRS001,-2149724.236,4410605.355,4061567.167#
```

4. Baseline RTK solving configuration

Parameter	Description
pos1-posmode	Solving mode — the fused positioning solving mode. Options: single: GNSS single-point solving dgps: GNSS differential solving static: static mode ppp-kine: GNSS precise kinematic single-point solving ppp-static: GNSS precise static single-point solving
pos1-frequency	Solving frequency
pos1-soltype	Solution type
pos1-elmask	Maximum satellite elevation angle
pos1-tidecorr	Tide correction: off, on
pos1-ionoopt	Ionospheric correction parameters: off: ionospheric correction off brdc: broadcast ionospheric correction model sbas: SBAS ionospheric correction model dual-freq: dual-frequency ionospheric correction model ionex-tec: IONEX ionospheric correction model qzs-brdc: QZS broadcast ionospheric correction model qzs-lex: QZS-LEX ionospheric correction model stec: STEC ionospheric correction model
pos1-tropopt	Tropospheric correction parameters: off: tropospheric correction off saas: SAAS tropospheric correction model sbas: SBAS tropospheric correction model est-ztd: estimate tropospheric zenith total delay est-ztdgrad: estimate tropospheric zenith total delay and gradient parameters ztd: tropospheric zenith total delay correction model
pos1-sateph	Ephemeris parameters: brdc: broadcast ephemeris precise: precise ephemeris
pos1-navsys	Satellite systems used in solving: 1:gps+2:sbas+4:glo+8:gal+16:qzs+32:comp
pos2-armode	Ambiguity fixing mode: 0:off, 1:continuous, 2:instantaneous, 3:fix-and-hold, 4:pppar, 5:pppar-ils, 6:wlnl, 7:tcar, 8:wlnlc, 9:tcarc
pos2-arthres	Ambiguity fixing threshold (ratio)
pos2-arlockcnt	AR lock count
pos2-arelmask	AR maximum elevation angle
pos2-arminfix	AR minimum fix count

pos2-elmaskhold	AR hold maximum elevation angle
pos2-slipthres	Cycle-slip detection threshold
stats-eratio, stats-eratio2, stats-errphase, stats-errphaseel, stats-errphasebl, stats-errdoppler, stats-stdbias, stats-stdiono, stats-stdtrop, stats-prnaccelh, stats-prnaccelv, stats-prnbias, stats-prniono, stats-prntrop, stats-prnpos, stats-clkstab	Random error model parameters
pos2-maxage	Age of RTK observation data

```
pos1-posmode      =kinematic  # (0:single, 1:dgps, 2:kinematic, 3:static, 4:movingbase, 5:fixed, 6:ppp-kine, 7:ppp-static, 8:ppp-fixed)
pos1-frequency    =l1         # (1:l1, 2:l1+l2, 3:l1+l2+l5, 4:l1+l5)
pos1-soltype      =forward    # (0:forward, 1:backward, 2:combined)
pos1-elmask       =20         # (deg)
pos1-snrmask_r    =off        # (0:off, 1:on)
pos1-snrmask_b    =off        # (0:off, 1:on)
pos1-snrmask_L1   =0,0,0,0,0,0,0,0,0
pos1-snrmask_L2   =0,0,0,0,0,0,0,0,0
pos1-snrmask_L5   =0,0,0,0,0,0,0,0,0
pos1-dynamics     =off        # (0:off, 1:on)
pos1-tidecorr     =off        # (0:off, 1:on, 2:otl)
pos1-ionoopt      =brdc       # (0:off, 1:brdc, 2:sbas, 3:dual-freq, 4:est-stec, 5:ionex-tec, 6:qzs-brdc, 7:qzs-lex, 8:stec)
pos1-tropopt      =saas       # (0:off, 1:saas, 2:sbas, 3:est-ztd, 4:est-ztdgrad, 5:ztd)
pos1-sateph       =brdc       # (0:brdc, 1:precise, 2:brdc+sbas, 3:brdc+ssrapc, 4:brdc+ssrcom)
pos1-exclsats     =C02        # (prn ...)
pos1-navsys       =61         # (1:gps+2:sbas+4:glo+8:gal+16:qzs+32:comp)
pos2-armode       =fix-and-hold # (0:off, 1:continuous, 2:instantaneous, 3:fix-and-hold)
pos2-gloarmode    =off        # (0:off, 1:on, 2:autocal)
pos2-bdsarmode    =on         # (0:off, 1:on)
```

Figure 2.1 Baseline RTK solving configuration

5. User configuration

Format: user name, password#

Example:

```
test1,password#
test2,password#
```

6. Reference station information configuration

Format: reference station name, specific address, city, district, ECEF-X, ECEF-Y, ECEF-Z, ITRF, flag

Example:

```
S009,XX Street Qingpu District Shanghai,Shanghai,Qingpu,31.12333298,121.07179362,55.9978,2014,0
S010,XX Street Jinshan District Shanghai,Shanghai,Jinshan,30.91804368,121.16882480,90.5689,2014,0
```

2.2 Running the system

Program interface:

```
cors-engine  [-o file]  [-t level]
```

Parameters:

- `-o` — path of the system configuration file
- `-t` — log output level (1: error, 2: warning, 3: info)

Example:

```
cors-engine.exe -o .\cors\conf\cors.conf -t 3
```

2.3 Virtual console

The virtual console connects to the continuously operating reference station and high-precision satellite positioning and navigation system in remote or local mode; commands can be issued to the running system, and the current operating status of the system can be queried through the virtual console.

1. Starting the system

1. Type `start`;
2. Press Enter to confirm — `cors server already start` is returned and the system starts running.

```
D:\tencent\work\src\cors\cmake-build-debug\cors-engine.exe -o D:\tencent\work\src\cors\conf\cors.conf -t 1 -s -p 9000
cors-engine>start
 start
cors server already start
cors-engine>
```

Figure 2.2 Starting the system

2. Adding a reference station

1. Type `addsource`;
2. Press Enter to confirm.

```
D:\tencent\work\src\cors\cmake-build-debug\cors-engine.exe -o D:\tencent\work\src\cors\conf\cors.conf -t 1 -s -p 9000
cors-engine>addsource A001 127.0.0.1 8002 test pwd A001
```

Figure 2.3 Adding a reference station

3. Deleting a reference station

1. Type `delsource`;
2. Press Enter to confirm.

```
D:\tencent\work\src\cors\cmake-build-debug\cors-engine.exe -o D:\tencent\work\src\cors\conf\cors.conf -t 1 -s -p 9000
cors-engine>delsource A001
```

Figure 2.4 Deleting a reference station

4. Loading the system configuration

1. Type `loadopt` ;
2. Press Enter to confirm.

```
D:\tencent\work\src\cors\cmake-build-debug\cors-engine.exe -o D:\tencent\work\src\cors\conf\cors.conf -t 1 -s -p 9000
cors-engine>loadopt .\cors.conf
```

Figure 2.5 Loading the system configuration

5. Displaying ephemeris information

1. Type `navidata` ;
2. Press Enter to confirm.

```
C:\Users\allansu\Desktop\cors\cmake-build-release\cors-engine.exe -o D:\tencent\work\src\cors\conf\cors.conf -t 1 -s -p 9000
cors-engine>navidata
 navidata

SAT   S IOD IOC FRQ A/A SVH                Toe                 Toc             Ttr/Tof L2C L2P
G01  OK   3   3   0   0 000 2022/12/25 03:59:44 2022/12/25 03:59:44 2022/12/25 04:28:40   1   0
G02   - 107 107   0   0 000 2022/12/25 02:00:00 2022/12/25 02:00:00 2022/12/25 04:28:40   1   0
G03  OK  79  79   0   1 000 2022/12/25 06:00:00 2022/12/25 06:00:00 2022/12/25 04:28:40   1   0
G04  OK  73 841   0   0 000 2022/12/25 06:00:00 2022/12/25 06:00:00 2022/12/25 04:28:40   1   0
G05   -  54  54   0   0 000 2022/12/25 02:00:00 2022/12/25 02:00:00 2022/12/25 04:28:40   1   0
G06   -  39  39   0   0 000 2022/12/24 22:00:00 2022/12/24 22:00:00 2022/12/25 04:28:40   1   0
G07   -  14  14   0   0 000 2022/12/24 21:59:44 2022/12/24 21:59:44 2022/12/25 04:28:40   1   0
G08   -  21  21   0   0 000 2022/12/25 02:00:00 2022/12/25 02:00:00 2022/12/25 04:28:40   1   0
G09   -  47  47   0   0 000 2022/12/24 22:00:00 2022/12/24 22:00:00 2022/12/25 04:28:40   1   0
G10  OK   8   8   0   0 000 2022/12/25 06:00:00 2022/12/25 06:00:00 2022/12/25 04:28:40   1   0
G11   - 189 189   0   0 000 2022/12/25 00:00:00 2022/12/25 00:00:00 2022/12/25 04:28:40   1   0
G12  OK  76  76   0   2 000 2022/12/25 06:00:00 2022/12/25 06:00:00 2022/12/25 04:28:40   1   0
G13   -   2   2   0   0 000 2022/12/25 01:59:44 2022/12/25 01:59:44 2022/12/25 04:28:40   1   0
G14   - 101 357   0   0 000 2022/12/25 02:00:00 2022/12/25 02:00:00 2022/12/25 04:28:40   1   0
G15   -   8   8   0   0 000 2022/12/25 00:00:00 2022/12/25 00:00:00 2022/12/25 04:28:40   1   0
G16  OK  72  72   0   0 000 2022/12/25 06:00:00 2022/12/25 06:00:00 2022/12/25 04:28:40   1   0
G17   -  16  16   0   0 000 2022/12/24 19:59:44 2022/12/24 19:59:44 2022/12/25 04:28:40   1   0
G18  OK 169 681   0   0 000 2022/12/25 04:00:00 2022/12/25 04:00:00 2022/12/25 04:28:40   1   0
G19   -  67  67   0   0 000 2022/12/24 20:00:00 2022/12/24 20:00:00 2022/12/25 04:28:40   1   0
G20   -  13  13   0   0 000 2022/12/25 00:00:00 2022/12/25 00:00:00 2022/12/25 04:28:40   1   0
G21  OK  81  81   0   0 000 2022/12/25 06:00:00 2022/12/25 06:00:00 2022/12/25 04:28:40   1   0
G22  OK  46  46   0   0 000 2022/12/25 06:00:00 2022/12/25 06:00:00 2022/12/25 04:28:40   1   0
G23  OK 229 997   0   0 000 2022/12/25 04:00:00 2022/12/25 04:00:00 2022/12/25 04:28:40   1   0
G24  OK  19  19   0   0 000 2022/12/25 04:00:00 2022/12/25 04:00:00 2022/12/25 04:28:40   1   0
G25  OK  46  46   0   0 000 2022/12/25 06:00:00 2022/12/25 06:00:00 2022/12/25 04:28:40   1   0
G26  OK  55  55   0   0 000 2022/12/25 06:00:00 2022/12/25 06:00:00 2022/12/25 04:28:40   1   0
G27   -   9   9   0   0 000 2022/12/25 01:59:28 2022/12/25 01:59:28 2022/12/25 04:28:40   1   0
```

Figure 2.6 Displaying ephemeris information

6. Displaying satellite observation information

1. Type `observ name -nf` , where *name* is the reference station name and *nf* is the number of frequencies;
2. Press Enter to confirm.

```
cors-engine>observ A005 -2
 observ A005 -2

      TIME(GPST)       SAT        P1(m)          P2(m)          L1(cyc)          L2(cyc)  D1(Hz)  D2(Hz) S1 S2 LLI
2022/12/25 04:30:21.00 G03 23420938.443 23420941.409 123078512.069  95905264.478     0.0     0.0 42 41 0 0
2022/12/25 04:30:21.00 G10 24408251.942 24408261.841 128266123.529  99947604.321     0.0     0.0 38 36 0 0
2022/12/25 04:30:21.00 G22 20396908.793 20396908.186 107186510.655  83521960.585     0.0     0.0 48 44 0 0
2022/12/25 04:30:21.00 G25 22097698.016 22097701.036 116124241.763  90486454.451     0.0     0.0 42 33 0 0
2022/12/25 04:30:21.00 G26 21661257.655 21661262.051 113831051.899  88698925.223     0.0     0.0 45 49 0 0
2022/12/25 04:30:21.00 G29 23803041.285 23803042.929 125085847.842  97469428.393     0.0     0.0 42 31 0 0
2022/12/25 04:30:21.00 G31 20342134.663 20342133.894 106898696.635  83297707.883     0.0     0.0 48 56 0 0
2022/12/25 04:30:21.00 G32 21003474.404 21003476.191 110373987.916  86005710.076     0.0     0.0 51 58 0 0
2022/12/25 04:30:21.00 C01 37931085.477 37931086.782 197517083.087 152732719.625     0.0     0.0 41 46 0 0
2022/12/25 04:30:21.00 C02 38260758.319 38260755.424 199233773.858 154060163.653     0.0     0.0 40 47 0 0
2022/12/25 04:30:21.00 C03 37575708.529 37575705.652 195666533.232 151301756.647     0.0     0.0 43 46 0 0
2022/12/25 04:30:21.00 C04 39056676.713 39056676.570 203378335.808 157264995.470     0.0     0.0 36 45 0 0
2022/12/25 04:30:21.00 C06 36009424.855 36009421.228 187510505.859 144994999.604     0.0     0.0 47 50 0 0
2022/12/25 04:30:21.00 C07 39048905.868 39048904.599 203338013.585 157233894.812     0.0     0.0 38 43 0 0
2022/12/25 04:30:21.00 C09 37039750.897 37039750.254 192875698.238 149143703.534     0.0     0.0 45 50 0 0
2022/12/25 04:30:21.00 C10 39795389.752 39795390.860 207225116.102 160239658.168     0.0     0.0 38 40 0 0
2022/12/25 04:30:21.00 C13 39933472.895 39933482.205 207944048.043 160795523.286     0.0     0.0 36 40 0 0
2022/12/25 04:30:21.00 C14 22589952.753 22589952.485 117631859.776  90960349.187     0.0     0.0 46 52 0 0
2022/12/25 04:30:21.00 C16 36139710.028 36139711.815 188188949.602 145519638.229     0.0     0.0 49 49 0 0
2022/12/25 04:30:21.00 C24 22021790.925        0.000 114672922.945         0.000     0.0     0.0 50  0 0 0
2022/12/25 04:30:21.00 C25 23023557.372        0.000 119890178.599         0.000     0.0     0.0 49  0 0 0
2022/12/25 04:30:21.00 C26 24809623.041        0.000 129190777.106         0.000     0.0     0.0 44  0 0 0
2022/12/25 04:30:21.00 C33 22128299.165        0.000 115228573.438         0.000     0.0     0.0 50  0 0 0
2022/12/25 04:30:21.00 C39 36002396.451        0.000 187473912.123         0.000     0.0     0.0 50  0 0 0
2022/12/25 04:30:21.00 C40 38851940.130        0.000 202312338.318         0.000     0.0     0.0 43  0 0 0
2022/12/25 04:30:21.00 C41 23855848.054        0.000 124223704.733         0.000     0.0     0.0 46  0 0 0
2022/12/25 04:30:21.00 R01 19594483.752 19594489.560 104744163.177  81467869.124     0.0     0.0 50 46 0 0
2022/12/25 04:30:21.00 R08 19942816.464 19942819.770 106792925.910  83060960.796     0.0     0.0 52 52 0 0
2022/12/25 04:30:21.00 R10 20949183.112        0.000 111670929.245         0.000     0.0     0.0 48  0 0 0
2022/12/25 04:30:21.00 R11 19403652.772 19403656.310 103686598.661  80646167.138     0.0     0.0 53 53 0 0
2022/12/25 04:30:21.00 R12 22705729.655 22705737.679 121290327.166  94337192.418     0.0     0.0 46 46 0 0
2022/12/25 04:30:21.00 E01 23564592.719 23564593.255 123832404.913  94884948.346     0.0     0.0 48 54 0 0
2022/12/25 04:30:21.00 E04 23712377.866 23712377.615 124609421.724  95479953.497     0.0     0.0 48 53 0 0
2022/12/25 04:30:21.00 E09 27742239.415 27742243.704 145786464.174 111706534.930     0.0     0.0 38 41 0 0
```

Figure 2.7 Displaying satellite observation information

7. Displaying satellite status information

1. Type `satellite name -nf`, where *name* is the reference station name and *nf* is the number of frequencies;
2. Press Enter to confirm.

```
cors-engine>satellite A005 -2
 satellite A005 -2

SAT C1    Az    El L1 L2  Fix1  Fix2  P1Res  P2Res   L1Res   L2Res  S11  S12  Lock1  Lock2 Rj1 Rj2
G02 OK 129.1 43.9  -  -     -     -  1.357  0.000  0.0000  0.0000    0    0      0      0   0   0
G05 OK   0.3 70.5  -  -     -     -  0.020  0.000  0.0000  0.0000    0    0      0      0   0   0
G07  -  44.9 10.9  -  -     -     -  0.000  0.000  0.0000  0.0000    0    0      0      0   0   0
G13 OK 158.8 67.6  -  -     -     -  0.801  0.000  0.0000  0.0000    0    0      0      0   0   0
G15 OK 213.1 45.8  -  -     -     - -0.788  0.000  0.0000  0.0000    0    0      0      0   0   0
G18 OK 309.7 16.6  -  -     -     - -1.127  0.000  0.0000  0.0000    0    0      0      0   0   0
G20 OK  62.7 49.9  -  -     -     -  0.601  0.000  0.0000  0.0000    0    0      0      0   0   0
G29 OK 267.2 49.2  -  -     -     - -0.937  0.000  0.0000  0.0000    0    0      0      0   0   0
G30 OK  76.5 21.6  -  -     -     -  1.444  0.000  0.0000  0.0000    0    0      0      0   0   0
R03 OK  34.0 38.1  -  -     -     - -3.283  0.000  0.0000  0.0000    0    0      0      0   0   0
R04 OK 299.3 71.8  -  -     -     -  2.922  0.000  0.0000  0.0000    0    0      0      0   0   0
R05 OK 240.2 27.8  -  -     -     - -2.932  0.000  0.0000  0.0000    0    0      0      0   0   0
R18 OK 157.7 43.1  -  -     -     - -1.732  0.000  0.0000  0.0000    0    0      0      0   0   0
R19 OK 289.3 83.8  -  -     -     -  0.774  0.000  0.0000  0.0000    0    0      0      0   0   0
R20 OK 329.0 22.3  -  -     -     -  4.250  0.000  0.0000  0.0000    0    0      0      0   0   0
E03 OK 208.1 69.8  -  -     -     -  0.809  0.000  0.0000  0.0000    0    0      0      0   0   0
E05 OK  50.9 52.1  -  -     -     - -0.860  0.000  0.0000  0.0000    0    0      0      0   0   0
E08 OK 221.2 17.3  -  -     -     -  1.024  0.000  0.0000  0.0000    0    0      0      0   0   0
E13  - 315.4 10.2  -  -     -     -  0.000  0.000  0.0000  0.0000    0    0      0      0   0   0
E15 OK 314.7 61.8  -  -     -     - -0.969  0.000  0.0000  0.0000    0    0      0      0   0   0
E36  - 135.7 10.3  -  -     -     -  0.000  0.000  0.0000  0.0000    0    0      0      0   0   0
J01 OK  88.7 71.1  -  -     -     - -0.252  0.000  0.0000  0.0000    0    0      0      0   0   0
J03 OK 160.6 37.5  -  -     -     - -1.035  0.000  0.0000  0.0000    0    0      0      0   0   0
J07 OK 162.8 42.8  -  -     -     - -0.853  0.000  0.0000  0.0000    0    0      0      0   0   0
C01 OK 139.4 35.6  -  -     -     - -0.621  0.000  0.0000  0.0000    0    0      0      0   0   0
C02 OK 225.0 34.8  -  -     -     - -0.601  0.000  0.0000  0.0000    0    0      0      0   0   0
C03 OK 188.6 45.9  -  -     -     - -0.037  0.000  0.0000  0.0000    0    0      0      0   0   0
```

Figure 2.8 Displaying satellite status information

8. Displaying reference station status information

1. Type `sourceinfo name`, where *name* is the reference station name; when *name* is `all`, the status information of all reference stations is displayed;
2. Press Enter to confirm.

```
cors-engine> start
start
cors server start
cors-engine> addsource test_zkw 182.254.57.107 8002 zkwntrip ******** RTCM32 40.1114 116.0935 90.0
addsource test_zkw 182.254.57.107 8002 zkwntrip ******** RTCM32 40.1114 116.0935 90.0

cors-engine> sourceinfo all
sourceinfo all

    Name   ID     Type              IP   Port     User      Password   Mountpoint      Status                                ECEF-XYZ/m
test_zkw    1    NTRIP   182.254.57.107   8002 zkwntrip      ********       RTCM32    connected    -2148524.377,4386943.549,4087511.314
```

Figure 2.9 Displaying reference station status information

9. Adding a virtual reference station

1. Type `addvsta name X Y Z`, where *name* is the virtual reference station name and X, Y, Z are the coordinates;
2. Press Enter to confirm.

```
D:\tencent\work\src\cors\cmake-build-debug\cors-engine.exe -o D:\tencent\work\src\cors\conf\cors.conf -t 1 -s -p 9000
cors-engine>addvsta VRS001 -2149724.236,4410605.355,4061567.167
```

Figure 2.10 Adding a virtual reference station

10. Deleting a virtual reference station

1. Type `delvsta name`, where *name* is the virtual reference station name;
2. Press Enter to confirm.

```
D:\tencent\work\src\cors\cmake-build-debug\cors-engine.exe -o D:\tencent\work\src\cors\conf\cors.conf -t 1 -s -p 9000
cors-engine>delvsta VRS001
```

Figure 2.11 Deleting a virtual reference station

11. Displaying the reference station triangular network

1. Type `showdtrigs`;
2. Press Enter to confirm.

```
D:\tencent\work\src\cors\cmake-build-debug\cors-engine.exe -o D:\tencent\work\src\cors\conf\cors.conf -t 1 -s -p 9000
cors-engine>showdtrigs
 showdtrigs
  1:      2->1->4
  2:      3->4->1
  3:      4->3->5
  4:      4->5->2
  5:      7->6->2
  6:      5->7->2
  7:      5->3->8
  8:      6->7->8
  9:      8->7->5
 10:      8->9->6
 11:      9->8->3
cors-engine>
```

Figure 2.12 Displaying the reference station triangular network

12. Displaying baseline RTK solution information

1. Type `showbls`;
2. Press Enter to confirm.

```
cors-engine>showbls
 showbls
2022/12/25 04:32:22.000:   1->  2 stat= FLOAT bl=    39 KM
2022/12/25 04:32:22.000:   1->  3 stat= FLOAT bl=    56 KM
2022/12/25 04:32:22.000:   1->  4 stat= FLOAT bl=    21 KM
2022/12/25 04:32:22.000:   4->  2 stat=   FIX bl=    20 KM
2022/12/25 04:32:22.000:   4->  3 stat= FLOAT bl=    45 KM
2022/12/25 04:32:22.000:   3->  5 stat= FLOAT bl=    24 KM
2022/12/25 04:32:22.000:   5->  4 stat= FLOAT bl=    28 KM
2022/12/25 04:32:22.000:   5->  2 stat= FLOAT bl=    31 KM
2022/12/25 04:32:22.000:   3->  6 stat= FLOAT bl=    72 KM
2022/12/25 04:32:22.000:   7->  5 stat=   FIX bl=    18 KM
2022/12/25 04:32:22.000:   2->  7 stat= FLOAT bl=    25 KM
2022/12/25 04:32:22.000:   3->  8 stat= FLOAT bl=    25 KM
2022/12/25 04:32:22.000:   8->  5 stat=   FIX bl=    14 KM
2022/12/25 04:32:22.000:   7->  8 stat=   FIX bl=    19 KM
2022/12/25 04:32:22.000:   8->  6 stat= FLOAT bl=    58 KM
2022/12/25 04:32:22.000:   7->  9 stat= FLOAT bl=    68 KM
2022/12/25 04:32:22.000:   9->  2 stat= FLOAT bl=    92 KM
2022/12/25 04:32:22.000:   6->  9 stat=   FIX bl=     0 KM
2022/12/25 04:32:22.000:   9->  8 stat= FLOAT bl=    58 KM
2022/12/25 04:32:22.000:   2-> -1 stat=   FIX bl=    19 KM
2022/12/25 04:32:22.000:   1-> -1 stat= FLOAT bl=    21 KM
2022/12/25 04:32:22.000:   4-> -1 stat=   FIX bl=     3 KM
```

Figure 2.13 Baseline RTK solution information

13. Displaying subnetworks

1. Type `showsubnet`;
2. Press Enter to confirm.

```
cors-engine>showsubnet
 showsubnet
  1:    2    3    4
  2:    1    4    5    6    7
  3:    1    4    5    8    9
  4:    1    2    3    5
  5:    3    4    2    7    8
  6:    2    7    8    9
  7:    5    6    2    8
  8:    3    5    7    6    9
  9:    8    6    3
```

Figure 2.14 Displaying subnetworks

14. Virtual reference station list

1. Type `showvsta`;
2. Press Enter to confirm.

```
cors-engine>showvstas
 showvstas
VRS001   -1: pos=-2149724.236 4410605.355 4061567.167
```

Figure 2.15 Virtual reference station list

15. Adding a user

1. Type `adduser name password`;
2. Press Enter to confirm.

```
cors-engine>adduser test pwd
 adduser test pwd
```

Figure 2.16 Adding a user

16. Deleting a user

1. Type `deluser name`;
2. Press Enter to confirm.

```
cors-engine>deluser test
 deluser test
```

Figure 2.17 Deleting a user

17. User list

1. Type `showusers`;
2. Press Enter to confirm.

```
D:\tencent\work\src\cors\cmake-build-debug\cors-engine.exe -o D:\tencent\work\src\cors\conf\cors.conf -t 1 -s -p 9000
cors-engine>showusers
 showusers
   test1 password
   test2 password
cors-engine>
```

Figure 2.18 User list

18. Displaying monitoring users

1. Type `moniinfo`;
2. Press Enter to confirm.

```
cors-engine>moniinfo
 moniinfo

          IP   MonitorSource
192.168.83.1            A001
192.168.83.1            A003
192.168.83.1            A005
cors-engine>
```

Figure 2.19 Monitoring users

19. NTRIPAGENT access user list

1. Type `agentinfo`;
2. Press Enter to confirm.

```
cors-engine> agentinfo
agentinfo

            IP Type   StationChg          User      Password      Mountpoint          Status                                ECEF-XYZ/m
   192.168.83.1    2           1         test1      password            A004      connected     -2224345.827,4345803.886,4090863.531
   192.168.83.1    2           1         test1      password            A004      connected     -2224345.827,4345803.886,4090863.531
   192.168.83.1    1           1         test1      password            A001      connected     -2224345.827,4345803.886,4090863.531
   192.168.83.1    1           1         test1      password            A003      connected     -2224345.827,4345803.886,4090863.531
   192.168.83.1    2           1         test1      password            A004      connected     -2224345.827,4345803.886,4090863.531
cors-engine>
```

Figure 2.20 NTRIPAGENT access user list

20. Drawing the Delaunay reference station network

1. Type `drawdtrignet dtrignet.png`;
2. Press Enter to confirm.

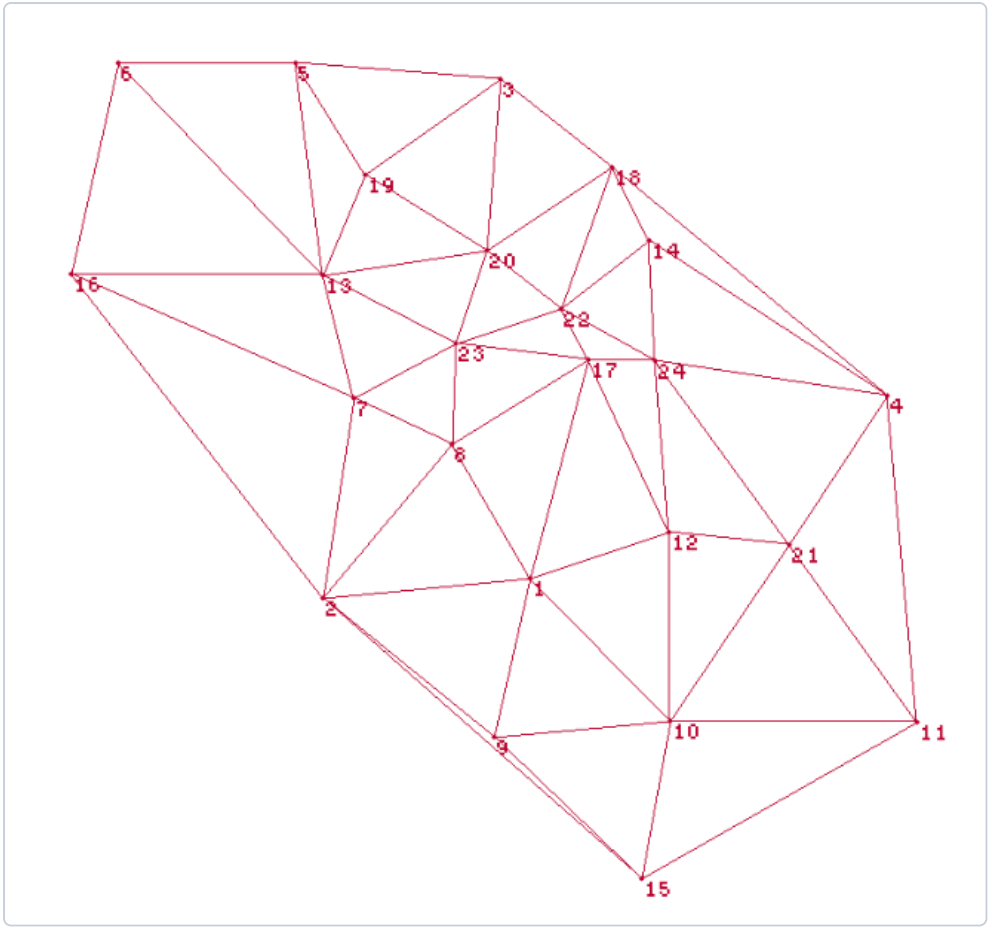


Figure 2.21 Delaunay reference station network

21. Drawing the position relationship between virtual and reference stations

1. Type `drawvsta VRS001 VRS001.png` ;
2. Press Enter to confirm.

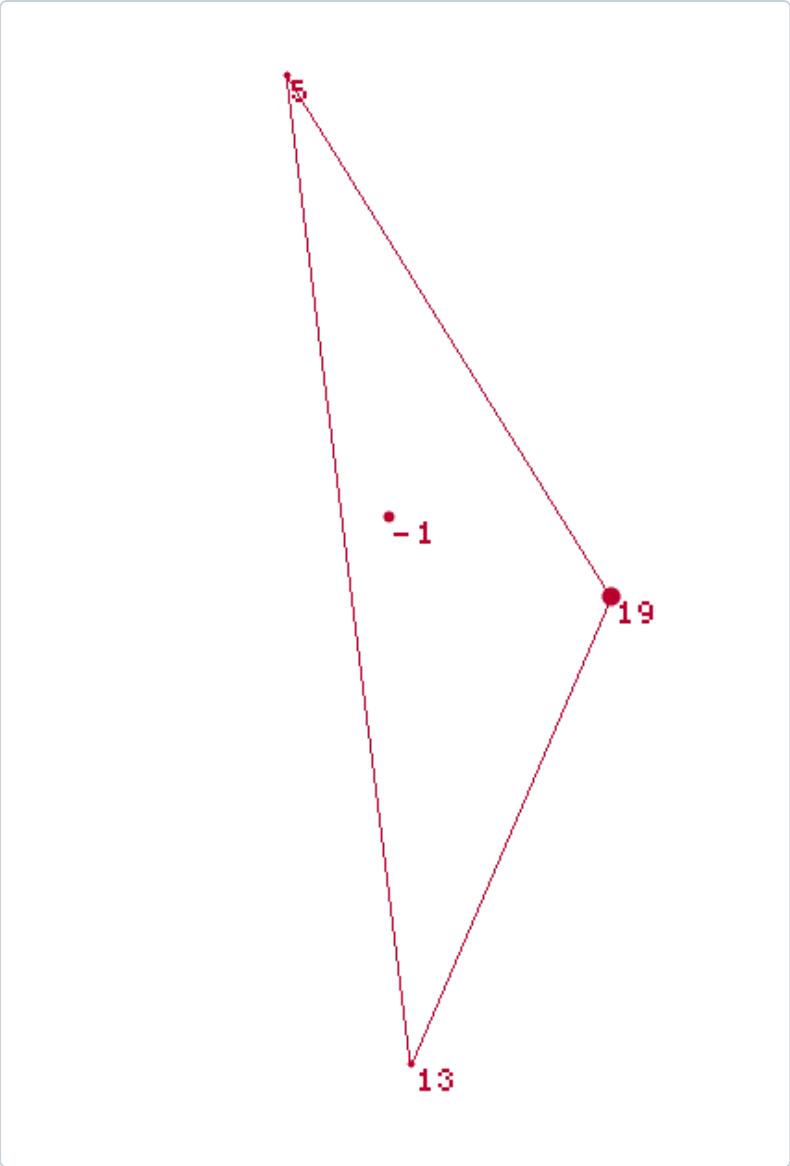


Figure 2.22 Position relationship between the virtual reference station VRS001 and the reference stations

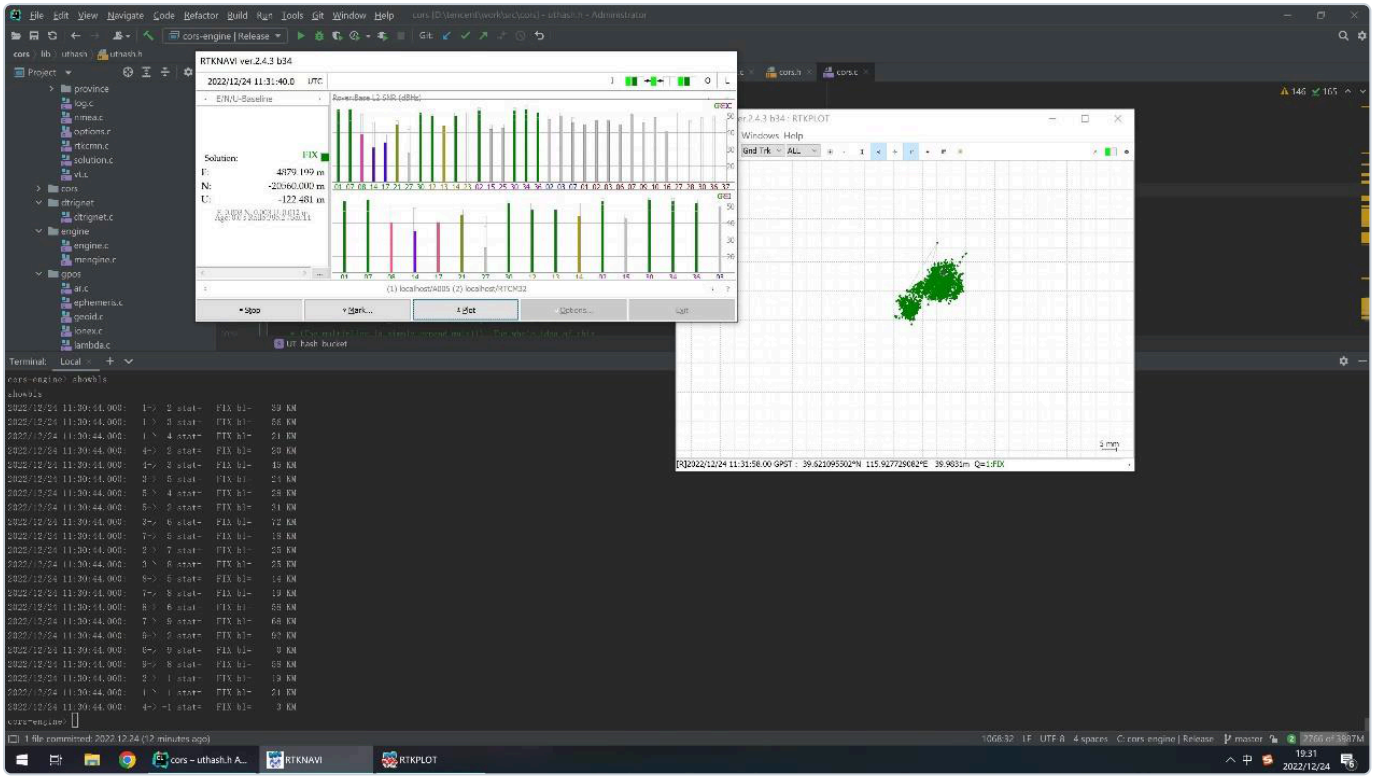
22. Stopping the system

1. Type `stop` — `stop cors server` is returned;
2. Press Enter to confirm.

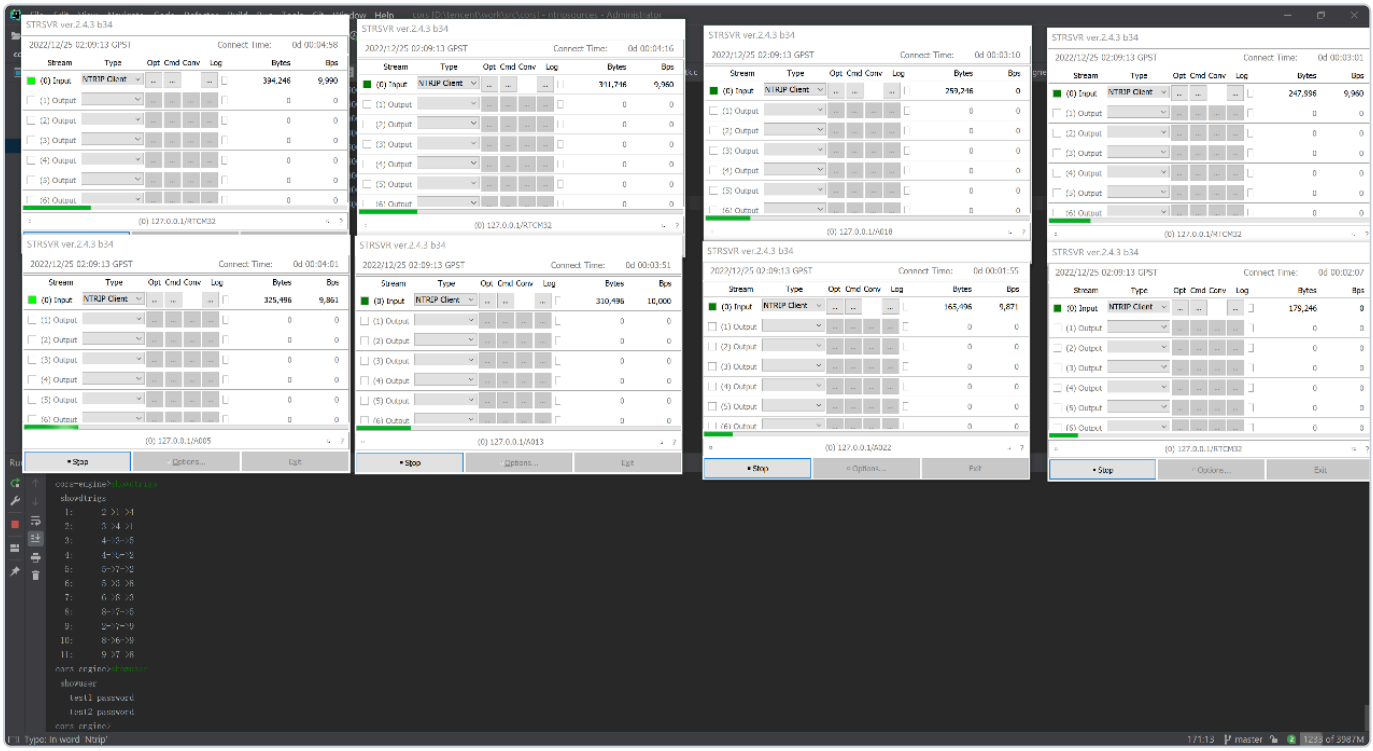
```
cors-engine>stop
 stop
stop cors server
cors-engine>
```

Figure 2.23 Stopping the system

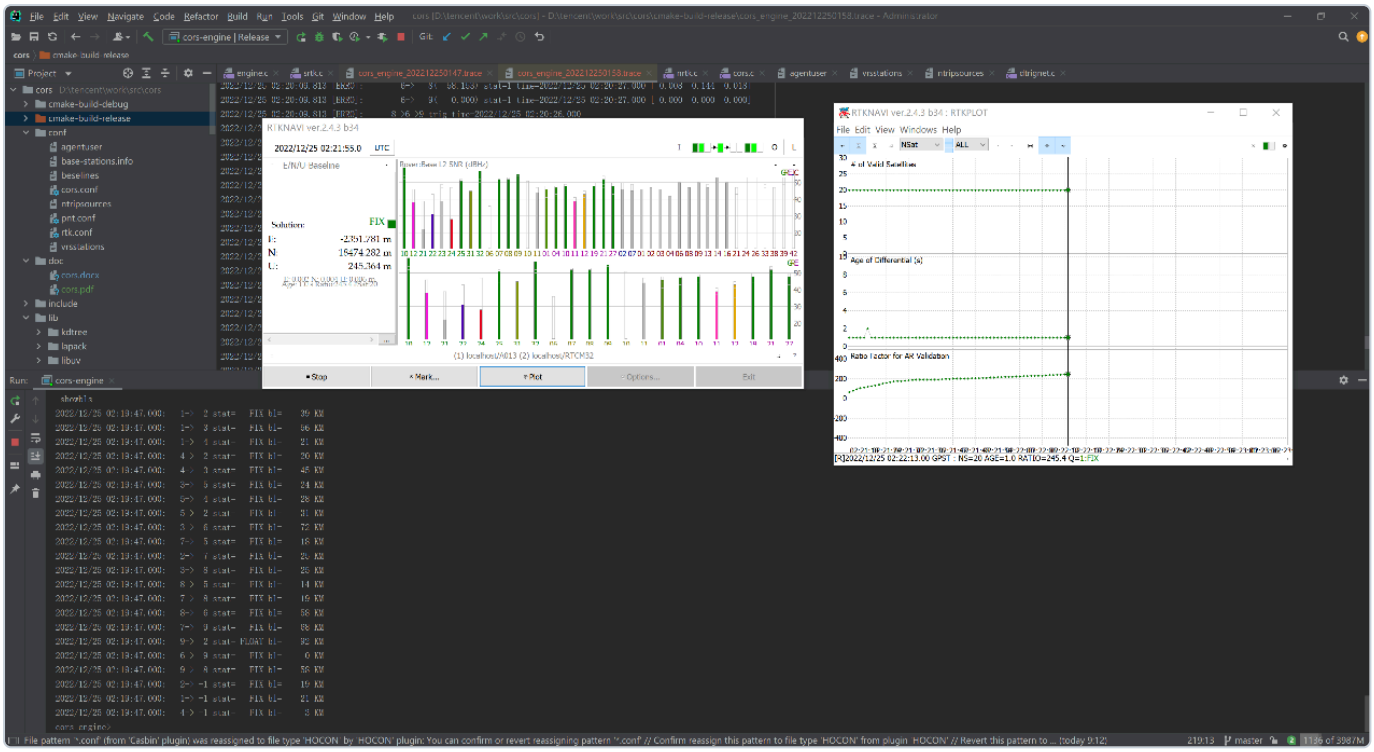
23. Runtime views



(a) Rover-side positioning test with the CORS solution stream (RTKNAVI / RTKPLOT)



(b) Real-time signal-to-noise monitoring of the reference station streams



(c) Engine trace logging and solution monitoring during operation

Figure 2.24 Runtime views

3 Project structure, code framework and algorithm flow

This chapter describes how the system is organized at the source-code level: the repository layout, the module layering and concurrency model of the code framework, and the internal algorithm flows of the key processing stages (Delaunay networking, baseline RTK solving and VRS observation generation). All descriptions refer to the source tree of the current release.

3.1 Project structure

The complete engine is written in C (C99) and organized as a flat set of functional modules under `src/`, roughly 36,400 lines of code in total. Third-party libraries are vendored under `lib/` so that the project builds without any external package manager. The top-level layout is:

```
cors-main/
├── CMakeLists.txt          Build definition (CMake ≥ 3.0)
├── include/                Public headers
│   ├── cors.h              Central data structures and module API
│   ├── rtklib.h            Positioning types and API (RTKLIB-derived)
│   ├── options.h           Configuration option table
│   ├── vt.h                Virtual terminal (console) API
│   ├── uve.h               libuv helper extensions
│   └── log.h               Logging API
├── src/                    Engine sources (see Table 3.2)
│   ├── common/             Shared utilities, options, solution output, NMEA, virtual terminal
│   ├── cors/               Core state container and life-cycle management
│   ├── engine/             Program entry point and interactive console
│   ├── dtrignet/           Incremental Delaunay reference-station networking
│   ├── nrtk/               Network RTK manager (baseline life cycle, subnetworks)
│   ├── vrs/                Virtual reference station observation generation
│   ├── gpos/               Positioning kernels: EKF RTK, LAMBDA, SPP, PPP, models
│   ├── rtcm/               RTCM 2/3 decoder and RTCM 3 MSM encoder
│   ├── ntrip/              NTRIP client, caster and agent (user service)
│   ├── monitor/            HTTP monitoring views (dashboard)
│   ├── rinex/              RINEX observation writer
│   ├── pnt/  ppp/          Thin service-side wrappers for point positioning / PPP
├── lib/                    Vendored third-party sources
│   ├── libuv/              Asynchronous event loop (threading and network I/O)
│   ├── kdtree/             k-d tree for nearest-vertex queries
│   ├── triangulator/       Incremental Delaunay triangulation
│   ├── uthash/  queue/     Hash tables and queue containers (header-only)
│   ├── lapack/             Dense linear algebra used by the estimators
│   └── draw/               Drawing helpers for console net plotting
├── test/                   Unit tests (Delaunay, RTCM codec, rtkpos, ...)
├── conf/                   Runtime configuration (cors.conf, rtk.conf, pnt.conf, ...)
├── bin/                    Pre-built executables and runtime DLLs
└── doc/                    Manuals and documentation figures
```

Table 3.1 Top-level directories

Directory	Contents

`include/`	Public headers shared by all modules; `cors.h` defines the central `cors_t` state container and the API of every subsystem.
`src/`	All engine sources, one sub-directory per functional module (Table 3.2).
`lib/`	Vendored third-party code: libuv (event loop), k-d tree, incremental triangulator, uthash/queue containers, LAPACK.
`test/`	Standalone unit-test programs, e.g. `test_dtrignet.c` , `test_rtcm_decoder.c` , `test_rtkpos.c` .
`conf/`	Configuration files read at start-up: `cors.conf` (system), `rtk.conf` (baseline RTK), `pnt.conf` (positioning service), `base-stations.info` , `ntripsources` , `agentuser` , etc.
`bin/`	Pre-built executables, runtime DLLs and the post-processing/replay package.
`doc/`	User manuals (PDF/HTML) and figures.

Table 3.2 Source modules (`src/`)

Module	Files	Lines	Responsibility
`engine`	2	908	Program entry (`main`), option loading, virtual-terminal console thread and the command interpreter (`start` , `addsource` , `addvsta` , `showbls` , ...).
`cors`	2	397	Global `cors_t` state container, life-cycle (`cors_start` / `cors_stop`), navigation-data initialization.
`ntrip`	4	1,384	NTRIP protocol stack: `ntripclient` (pulls streams from physical stations, auto-reconnect, periodic GGA), `ntripcaster` (sourcetable and stream mounting), `ntripagent` (user authentication and access service).
`rtcm`	6	6,631	RTCM 2/3 message decoding and RTCM 3 MSM encoding (1001–1004, 1074/1076/1077 series, ephemerides, station coordinates).
`dtrignet`	1	421	Incremental Delaunay networking over reference stations: k-d tree lookup, triangle insertion/removal, edge add/delete queues.
`nrtk`	2	1,151	Network RTK manager: maps Delaunay edges to per-baseline solvers (`srtk`), applies add/delete/update requests, maintains subnetworks.
`vrs`	1	658	Virtual reference station engine: triangle location, master-station selection, interpolation coefficients, VRS observation synthesis, RTCM/RINEX output.
`gpos`	12	14,278	Positioning kernels: extended Kalman filter RTK (`rtkpos`), LAMBDA ambiguity resolution, single-point positioning (`pntpos`), PPP, ephemeris and satellite clock handling, precise orbit interpolation (`preceph`), troposphere/ionosphere models, tides, SBAS, quasi-geoid grids (`geoid`).
`monitor`	10	1,400	HTTP monitoring views on the monitor port: base-station information and distribution, navigation data, RTCM message statistics, source table.
`rinex`	1	2,973	RINEX observation writer, used for VRS observation files and logging.
`common`	6	6,016	Shared foundation: vector/matrix and time utilities (`rtkcmn`), configuration options, solution formatting, NMEA parsing, virtual terminal (`vt`), logging.

`pnt` / `ppp`	2	163	Thin service-side wrappers that expose the `gpos` kernels to the CORS service chain.

3.2 Code framework

3.2.1 Module layering

The code is organized in five layers with strictly downward dependencies, as shown in Figure 3.1. The service-interface layer talks to the outside world (console operators, rovers, physical stations, monitoring dashboards); the dispatch layer owns the global state and routes events between interfaces and kernels; the processing kernels implement the GNSS algorithms; the positioning-primitive layer provides the mathematical estimation machinery; and the foundation layer contains shared utilities and vendored third-party libraries used by everything above.

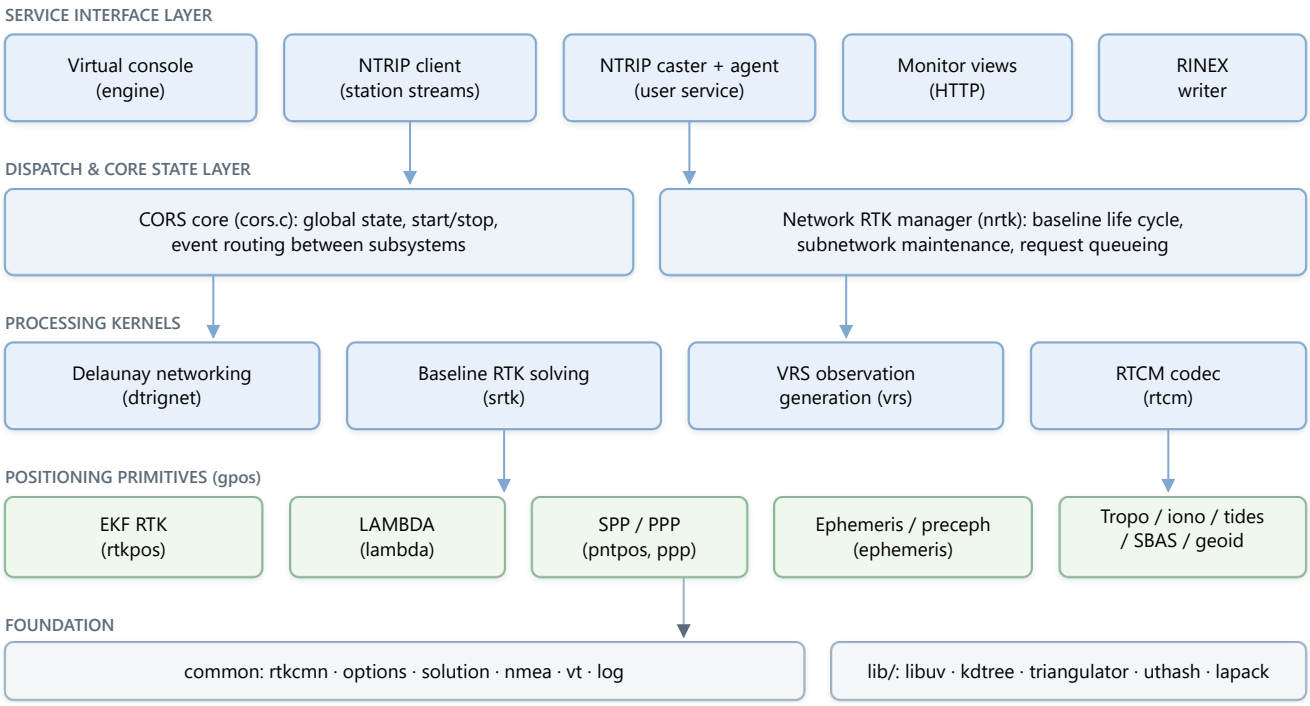


Figure 3.1 Module layering of the code framework

3.2.2 Threading and concurrency model

The engine is fully asynchronous. Every subsystem runs in its own operating-system thread and owns a private **libuv** event loop; the threads never share mutable state directly. Instead, each thread exposes a small set of **uv_async** handles that act as cross-thread message queues: a producer wakes the target loop with `uv_async_send` and hands over a heap-allocated request, which the owner thread dequeues and executes on its own loop. This design gives lock-free handoff between stages, keeps every data structure single-owner, and makes the system robust to slow or failing network peers. Figure 3.2 shows the thread topology and the message channels.

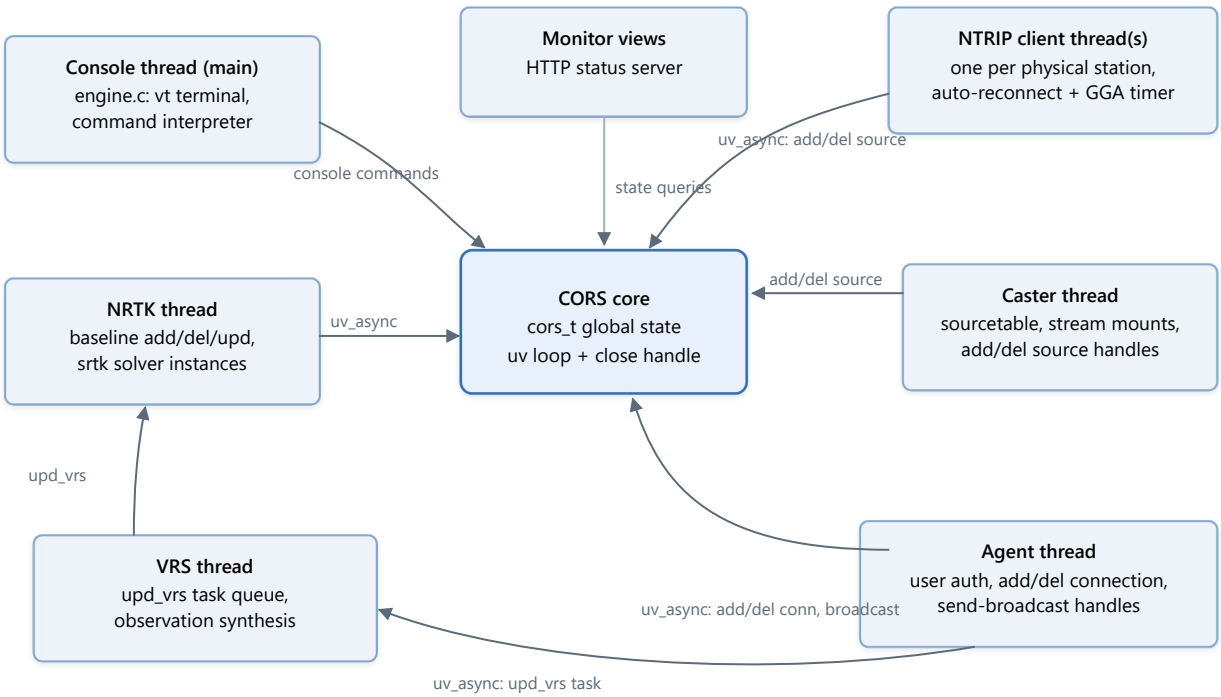


Figure 3.2 Threading and concurrency model: one thread and one libuv loop per subsystem, cross-thread communication through uv_async message queues

Grey arrows are uv_async message channels between threads; every box owns one private libuv event loop, and requests are handed over as heap-allocated messages so that no mutable state is shared across threads.

3.2.3 Core data structures

Everything hangs off a single global instance `cors_t` (declared in `include/cors.h`). The important structures and their relationships are summarized in Table 3.3.

Table 3.3 Core data structures

Structure	Defined in	Role
`cors_t`	`cors.h`	Root container: options, navigation data, the NTRIP stack, the NRTK manager, the VRS engine and the monitor; owns start/stop life cycle.
`cors_dtrig_net_t`	`cors.h`	Delaunay network: hash table of vertices, kd-tree index, triangle set, add/delete edge queues fed to NRTK.
`cors_dtrig_t`	`cors.h`	One triangle: three vertex pointers, containing-subnetwork link, per-frequency master-station data.
`cors_nrtk_t`	`cors.h`	Network RTK manager: request queue (add/del source, add/del vsta, add/del/upd baseline), subnetwork definitions, per-baseline solver table.
`cors_srtk_t` / `cors_baseline_t`	`cors.h`	One single-baseline RTK solver per Delaunay edge, holding the `rtk_t` estimator state and the latest solution (FIX/FLOAT).

`cors_vrs_t` / `cors_vrs_sta_t`	`cors.h`	VRS engine: virtual-station table with position, containing triangle, master station, generated observations and output tasks.
`rtk_t`	`rtklib.h`	Estimator state: Kalman filter vector/covariance, per-satellite slip/fix/lock flags, solutions — one instance per baseline and per master-to-VRS pair.
`obs_t` / `nav_t`	`rtklib.h`	Observation epochs and ephemeris/clock storage shared by all solving stages.

3.3 Key algorithm flows

3.3.1 End-to-end data flow

Figure 3.3 traces one observation epoch through the system. Streams pulled from the physical stations are decoded into observations and ephemerides; the Delaunay network turns the station set into baselines solved by per-edge RTK engines; the VRS engine combines fixed baselines with a user request to synthesize virtual observations; and the RTCM encoder hands the result back to the caster, which broadcasts it to the rover. The monitor samples the same internal state for the dashboard.

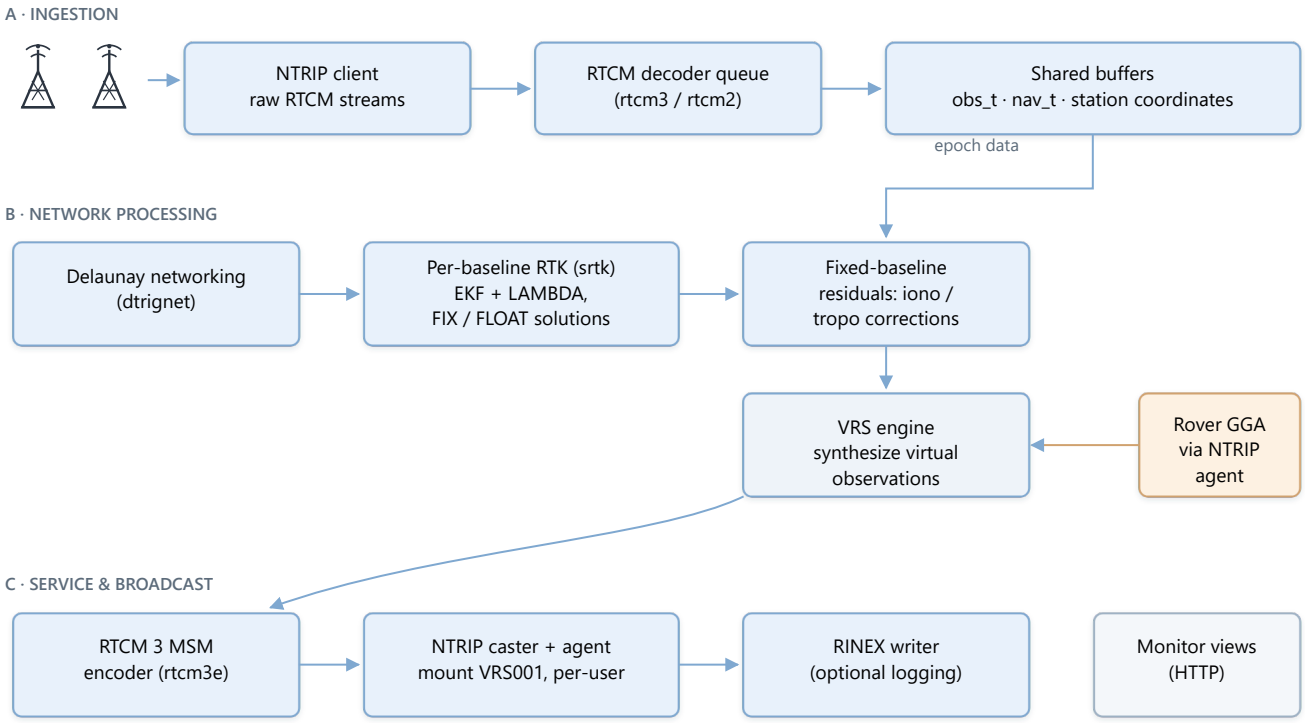


Figure 3.3 End-to-end data flow from reference stations to rovers

3.3.2 Incremental Delaunay networking

Reference stations are added and removed dynamically, so the triangulation is maintained **incrementally** rather than rebuilt. When a station is added, the k-d tree locates its position in the existing mesh, the incremental triangulator (Bowyer–Watson style) re-triangulates only the affected region, and the resulting edge changes are placed on two queues

— added edges and deleted edges. The NRTK manager consumes these queues: every added edge creates a single-baseline solver, every deleted edge destroys one, and the subnetwork assignment of each triangle is refreshed. Figure 3.4 summarizes the sequence and illustrates a vertex insertion.

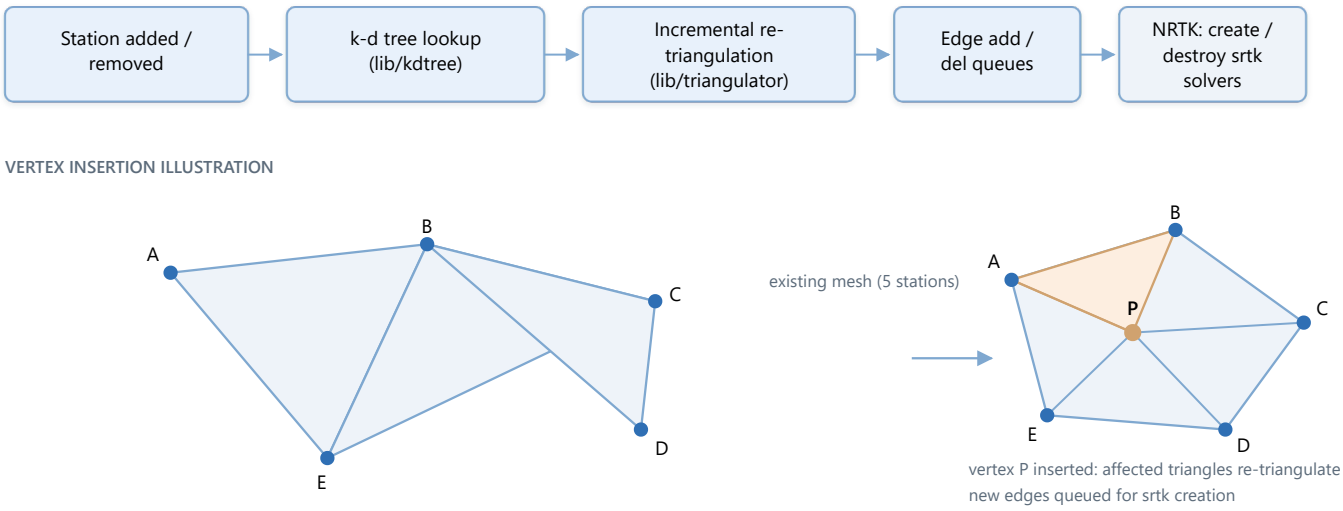


Figure 3.4 Incremental Delaunay networking and vertex insertion

3.3.3 Baseline RTK solving

Every Delaunay edge owns one single-baseline solver built on the extended Kalman filter in `gpos/rtkpos.c`. The state vector contains rover position and velocity, per-satellite ionospheric and tropospheric delays and per-satellite-per-frequency double-difference ambiguities. Each epoch runs the cycle shown in Figure 3.5: cycle-slip screening precedes the time update; double-difference residuals with MAD-based outlier screening feed the measurement update; and the ambiguity resolution stage applies the LAMBDA method with a ratio test whose threshold is adjusted by the number of fixed ambiguities. Fixed solutions additionally trigger partial ambiguity restoration (`restamb`) and, when enabled, ambiguity holding (`holdamb`) to reinforce subsequent fixes.

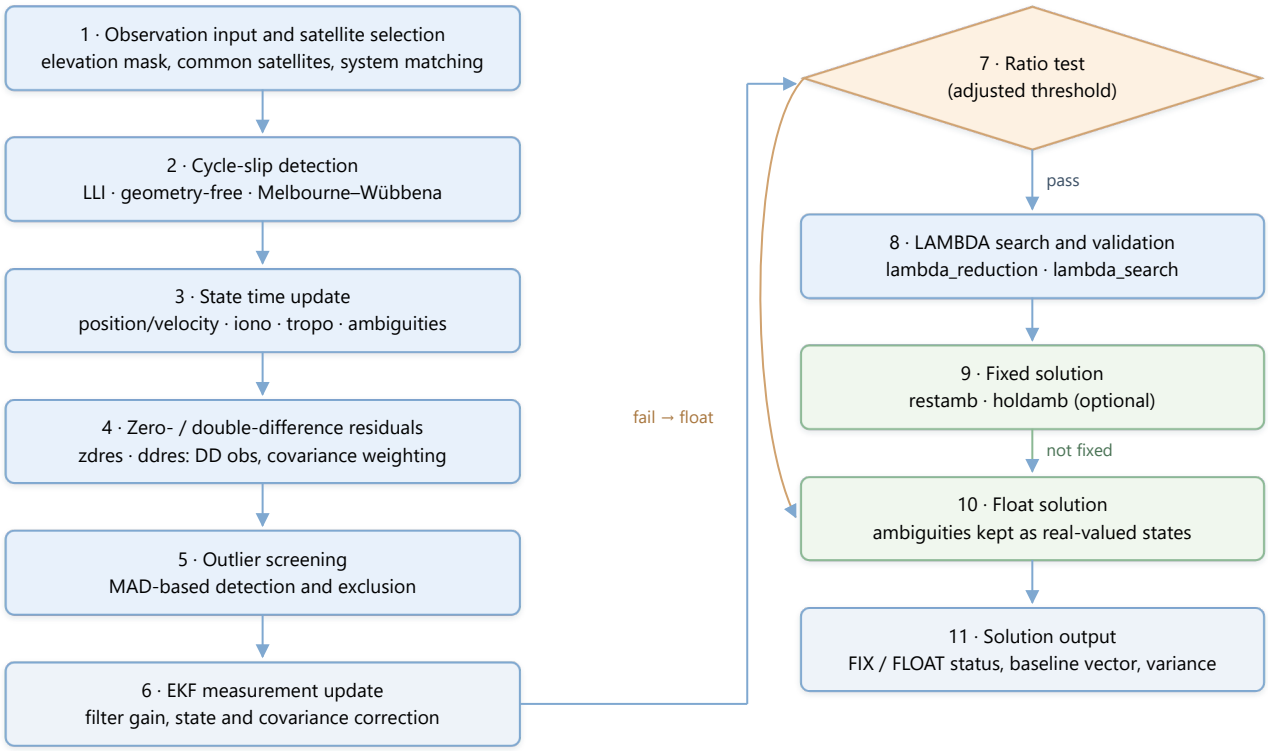


Figure 3.5 Baseline RTK solving flow (`gpos/rtkpos.c`)

3.3.4 VRS observation generation

When a rover requests a virtual reference station, the VRS engine locates the requested position inside the Delaunay mesh, picks the **master station** — the triangle vertex nearest to the request — and synthesizes a complete observation set at the virtual position. The atmospheric correction at the virtual position is obtained by least-squares interpolation of the corrections implied by the **ambiguity-fixed** baselines that involve the master station: each fixed baseline contributes one ENU-projected residual, and the solver fits a three-dimensional linear coefficient vector over them (a weakly-regularized row keeps the system determined when fewer than three baselines qualify). The virtual observation is then assembled from the master observation plus the geometric, clock, tropospheric and interpolated atmospheric terms shown in Figure 3.6, and finally encoded as RTCM 3 MSM (or written to RINEX). Tasks are queued as uv_async messages, so synthesis for many simultaneous users is pipelined on the VRS thread.

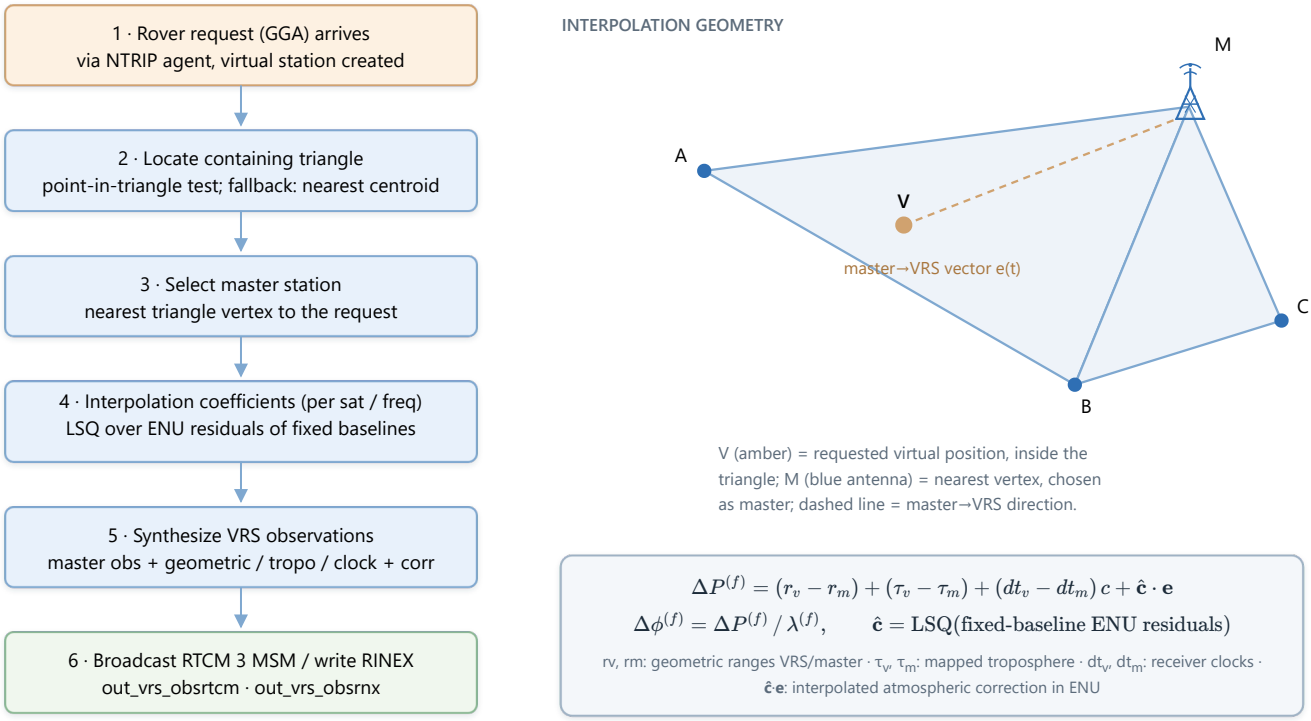


Figure 3.6 VRS observation generation: request flow (left) and master-station interpolation geometry (right)

3.4 Build system and third-party components

The project is built with CMake. The engine links against LAPACK (with its BLAS, gfortran and quadmath runtime dependencies) for the least-squares and Kalman-filter linear algebra; on Linux the build additionally requires `sqlite3`, `pthread` and `libm`. All remaining third-party components are vendored under `lib/` and compiled together with the engine, as listed in Table 3.4. Unit tests under `test/` cover the Delaunay networking, the RTCM codec round-trip, the RTK position engine and the nearest-station lookup, and are wired into the build through `test/CMakeLists.txt`.

Table 3.4 Vendored third-party components

Component	Location	Used for
libuv	`lib/libuv`	Event loops, asynchronous network I/O, cross-thread async message queues, timers.
k-d tree	`lib/kdtree`	Nearest-vertex and point-location queries in the Delaunay network.
Triangulator	`lib/triangulator`	Incremental Delaunay triangulation over reference stations.
uthash	`lib/uthash`	Header-only hash tables for stations, users, baselines and triangles.
queue	`lib/queue`	Intrusive queue containers used by the message plumbing.
LAPACK	`lib/lapack`	Dense linear algebra: least squares (VRS coefficients), LAMBDA reduction/search, filter updates.

Note: the positioning kernels in `gpos/` descend from the RTKLIB code base and have been extensively adapted for network operation — most notably the multi-constellation state handling, MAD outlier screening, the adjusted ratio-test threshold and the interfaces that expose per-baseline states to the VRS interpolator.